

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test parity generator and checker for bit error detection.	
Experiment No: 08	Date: 27-03-2026	Enrolment No: 92301733056

AIM: To test parity generator and checker for bit error detection.

IDE: EDA Playground

THEORY:

Parity checking is a widely used **error detection technique** in digital communication and computer systems. It helps in identifying whether the transmitted data has been altered during transmission due to noise or other disturbances.

In this method, an additional bit known as the **parity bit** is added to the original data bits. Depending on the type of parity used, the total number of 1's in the data (including the parity bit) is maintained as either even or odd.

DESIGN AND IMPLEMENTATION:

A. Parity Generator Module

A 4-bit even parity generator calculates the XOR of all bits in the input. This ensures that the number of 1's (including the parity bit) is even.

B. Parity Checker Module

Takes the same 4-bit data and the received parity bit. If the combined XOR results in 1, an error is present.

C. Testbench

Simulates various input cases and displays whether errors are correctly detected.

VERILOG CODE:

A. Parity Generator Module

```
// parity generator
module parity_generator(
    input [3:0] data,
    output parity
);
    assign parity = ^data; //xor of all bits
endmodule
```

B. Parity Checker Module

```
//parity checker
module parity_checker(
    input [3:0] data,
    input parity,
    output error
);
    assign error = ^{data, parity};
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test parity generator and checker for bit error detection.	
Experiment No: 08	Date:27-03-2026	Enrolment No: 92301733056

```
//error = 0 then no error is detected
//error = 1 then error is detected
endmodule
```

C. Testbench

```
`timescale 1ns/1ps
module tb_parity;
  reg [3:0] data;
  reg parity_in;
  wire parity_out;
  wire error;

  // name port association
  parity_generator pg(
    .data(data),
    .parity(parity_out)
  );

  parity_checker pc(
    .data(data),
    .parity(parity_in),
    .error(error)
  );

  initial begin
    $dumpfile("dump.vcd");
    $dumpvars(0,tb_parity);

    data = 4'b1010;
    #20 parity_in=parity_out;// correct parity(no error)

    data =4'b1010;
    #20 parity_in= ~parity_out;// incorrect parity(error introduced)

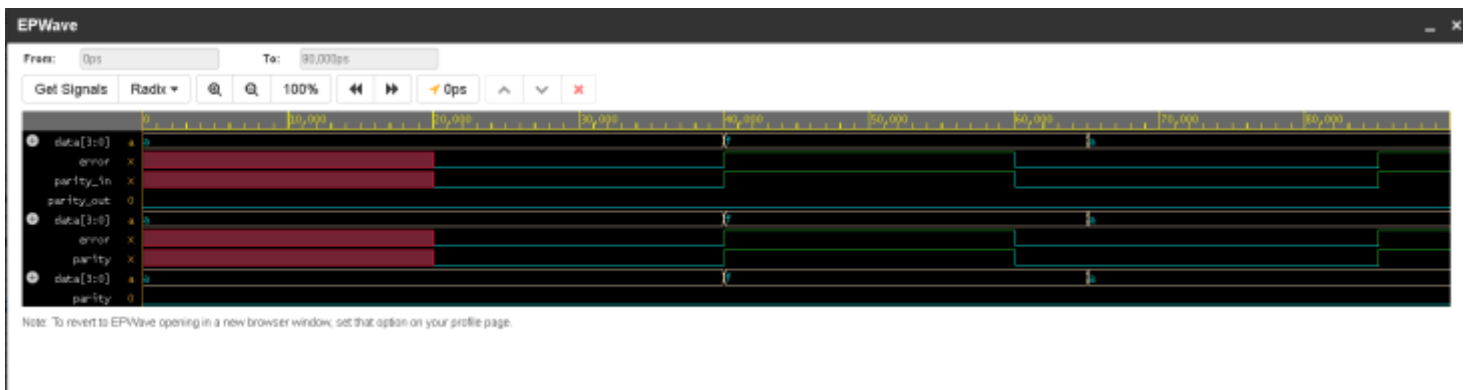
    data =4'b1111;
    #20 parity_in=parity_out;// correct parity(no error)
    #5;
    data =4'b1010;
    #20 parity_in=~parity_out;// incorrect parity(error introduced)
    #5;
    $finish;
  end

  initial begin
    $monitor("Time=%t | Data =%b | parity_in = %b |error =%b", $time, data, parity_out, parity_in, error);

  end
endmodule
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test parity generator and checker for bit error detection.	
Experiment No: 08	Date:27-03-2026	Enrolment No: 92301733056

SIMULATION RESULTS:



CONCLUSION:

In this experiment, the parity generator and parity checker were successfully designed and implemented using Verilog. The parity generator correctly produced the parity bit using XOR operation, ensuring even parity for the given data. The parity checker verified the received data and accurately detected errors by comparing the data bits with the parity bit. From the simulation results, it is observed that when the correct parity bit is provided, no error is detected, whereas an incorrect parity bit results in error detection. This confirms that the parity method is effective for identifying single-bit errors.

Thus, parity generation and checking provide a simple and efficient technique for basic error detection in digital communication systems.

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test parity generator and checker for bit error detection.	
Experiment No: 08	Date: 27-03-2026	Enrolment No: 92301733056

Post Lab Exercise:

To test parity generator and checker for bit error detection (For Odd Parity).

In odd parity, the parity bit is added such that the total number of 1's (including parity bit) is odd.

- If the number of 1's in data is even $\rightarrow$ parity bit = 1
- If the number of 1's in data is odd $\rightarrow$ parity bit = 0

This ensures that overall count of 1's becomes odd.

Design code:

```
// Code your design here
module parity_odd (
    input [3:0] data_in,
    input parity_in,
    output parity_bit,
    output error
);

// Odd parity generation
assign parity_bit = ~(data_in[0] ^ data_in[1] ^ data_in[2] ^ data_in[3]);

// Parity checking
assign error = ~(data_in[0] ^ data_in[1] ^ data_in[2] ^ data_in[3] ^ parity_in);

endmodule
```

Testbench Code:

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module parity_odd_tb;

reg [3:0] data_in;
reg parity_in;
wire parity_bit;
wire error;

// Instantiate the module
parity_odd uut (
    .data_in(data_in),
    .parity_in(parity_in),
```

 Marwadi University	Marwadi University Faculty of Technology Department of Information and Communication Technology	
Subject: Digital VLSI Testing (01CT0629)	Aim: To test parity generator and checker for bit error detection.	
Experiment No: 08	Date:27-03-2026	Enrolment No: 92301733056

```

.parity_bit(parity_bit),
.error(error)
);

initial begin
    // Dump waveform file
    $dumpfile("parity.vcd");
    $dumpvars(0, parity_odd_tb);

    // Test cases
    data_in = 4'b0000; parity_in = 1'b1; #10;
    data_in = 4'b0001; parity_in = 1'b0; #10;
    data_in = 4'b0011; parity_in = 1'b1; #10;
    data_in = 4'b0101; parity_in = 1'b0; #10;

    // Introducing error
    data_in = 4'b0111; parity_in = 1'b0; #10; // wrong parity

    data_in = 4'b1111; parity_in = 1'b1; #10;

    $finish;
end

endmodule

```

SIMULATION Result:

